



3.3. Replacing & Mapping Values

Previous Section:

 [3.2. Sorting Data Values](#)

Contents:

[1. Replacing Values](#)

[1.1. Replace a Single Value](#)

[1.2. Replace Multiple Values](#)

[2. Mapping Values](#)

[2.1. Basic Mapping](#)

[2.1. Label Encoding Example](#)

[2.2. Mapping with `unique\(\)`](#)

[3. Replacing Missing Values](#)

[3.1. Standardizing Missing Values](#)

[3.2. Imputing Missing Values](#)

[Summary](#)

[References](#)

So far, we have learned how to clean, filter, and sort data. Another common task in data analysis is **changing existing values** into more meaningful or standardized ones.

Pandas provides two closely related methods for this purpose:

- **Replacing** changes one or more existing values with new values.

- **Mapping** converts values according to a predefined relationship, making it especially useful for label encoding and feature transformation.

Although they may appear similar, they are designed for slightly different use cases.

We continue using the synthetic dataset below.

```
import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))
```

1. Replacing Values

Replacing is the simplest way to substitute one value with another.

The `replace()` method works on both **Series** and **DataFrames**, making it useful for correcting spelling mistakes, renaming categories, or standardizing inconsistent values.

1.1. Replace a Single Value

```
df["Major"] = df["Major"].replace("CSC", "Computer Science")
print(df.head())
```

1.2. Replace Multiple Values

A dictionary can replace several values at once.

```
df["Major"] = df["Major"].replace({
    "AI": "Artificial Intelligence",
    "IT": "Information Technology",
    "CSC": "Computer Science",
    "SW": "Software Engineering"
})
```

You can even replace values across the entire DataFrame if needed.

```
df.replace({"CSC": "CS", "SW": "SE"})
```

When should you use `replace()` ?

- Correcting incorrect values
- Standardizing labels
- Cleaning inconsistent data
- Replacing multiple categories simultaneously

2. Mapping Values

Mapping is similar to replacing, but it is designed specifically for **one-to-one value transformations**.

It is commonly used for:

- Label encoding
- Creating numeric categories
- Converting abbreviations into descriptive labels

Unlike `replace()`, `map()` **only works on a Series (a single column)**.

2.1. Basic Mapping

```
major_map = {  
    "AI": "Artificial Intelligence",  
    "IT": "Information Technology",  
    "CSC": "Computer Science",  
    "SW": "Software Engineering"  
}  
  
df["Major"] = df["Major"].map(major_map)
```

2.1. Label Encoding Example

Mapping is frequently used to convert text labels into numbers.

```
major_code = {
    "AI": 0,
    "IT": 1,
    "CSC": 2,
    "SW": 3
}

df["Major_Code"] = df["Major"].map(major_code)
```

You can also drop the "Major" column with `df_map.drop(columns="Major")`

2.2. Mapping with `unique()`

Sometimes you want to automatically assign numbers to every unique value.

```
unique_major = df["Major"].unique()

mapping = {
    value: index
    for index, value in enumerate(unique_major)
}

df["Major_Code"] = df["Major"].map(mapping)
```

This approach is convenient when you do not know all possible categories beforehand.



Note: If a value does not exist in the mapping dictionary, `map()` returns `NaN`. In contrast, `replace()` simply leaves unmatched values unchanged.

3. Replacing Missing Values

In real-world datasets, missing values rarely appear in just one form. Besides the standard `NaN`, missing data may also be represented as: `NaN` ; `None` ; `"N/A"` ; `"Unknown"` ; An empty string `""` ; A blank space `" "` ; A placeholder value such as `0` (when `0` is not a valid value)

Before we can impute (fill) missing values, we often need to **standardize all of these representations into a single format**, usually `NaN`.

For example, consider the following dataset containing multiple types of missing values.

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "Name": [
        "James", "Chris", "Adam", "Marian", "Peter",
        "Kevin", "Linda", "Emma", "Sophia", "Daniel"
    ],
    "Major": [
        "AI", "IT", "Unknown", "CSC", None,
        "", "SW", "N/A", "AI", "CSC"
    ],
    "Age": [
        19, 20, 0, 18, np.nan,
        19, None, 20, 18, 0
    ],
    "GPA": [
        3.60, np.nan, 2.90, 0,
        3.10, None, "N/A", 3.80,
        "", 3.20
    ]
})

df.set_index("Name", inplace=True)

print(df)
```

Notice that missing values appear in several different forms.

Column	Missing representations
Major	None, "", "Unknown", "N/A"
Age	0, None, NaN
GPA	NaN, None, "N/A", 0, ""

3.1. Standardizing Missing Values

We can replace all placeholder values with `NaN` before performing any imputation.

```
df = df.replace({
    "Unknown": np.nan,
    "N/A": np.nan,
    "": np.nan,
    " ": np.nan,
    0: np.nan
})
```

Now every missing value is represented consistently as `NaN`, making later processing much easier.

3.2. Imputing Missing Values

Once the missing values have been standardized, we can fill them using `fillna()`.

- For example, replace missing GPAs with the column mean:

```
df["GPA"] = df["GPA"].fillna(df["GPA"].mean())
```

- Replace missing majors with `"Undeclared"`:

```
df["Major"] = df["Major"].fillna("Undeclared")
```

- Replace missing ages with the median age:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Using `replace()` together with `fillna()` is a common workflow:

1. Convert every placeholder into `NaN`.
2. Apply an appropriate imputation strategy, such as the mean, median, mode, or a custom value.



Note: Although `0` can sometimes indicate missing data, it should only be replaced if `0` is **not a valid value** for that feature.

- For example, an age of `0` is usually invalid for university students, but a GPA of `0.0` may be a legitimate score depending on the grading system.

Summary

- `replace()` substitutes one or more values in a **Series or DataFrame**, making it useful for correcting errors, standardizing labels, and cleaning inconsistent data.
- `replace()` can replace a single value or multiple values simultaneously using a dictionary.
- `map()` performs **one-to-one value transformations** on a **Series** and is commonly used for label encoding and converting categorical values into meaningful labels or numeric codes.
- Unlike `replace()`, `map()` returns `NaN` for values that are not found in the mapping dictionary.
- `unique()` can be combined with `map()` to automatically create mappings for all unique values in a column.
- Missing values may appear in many forms, including `NaN`, `None`, `"N/A"`, `"Unknown"`, empty strings (`" "`), or placeholder values such as `0`.

- A common data-cleaning workflow is to first use `replace()` to convert all missing-value representations into `NaN`, then use `fillna()` to impute them with appropriate values such as the **mean**, **median**, **mode**, or a custom replacement.
- Together, `replace()`, `map()`, and `fillna()` provide a simple yet powerful toolkit for transforming, standardizing, and preparing data for analysis and machine learning.

To practice further, complete Lab 03 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.replace.html>

<https://www.geeksforgeeks.org/data-analysis/python-pandas-dataframe-replace/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.map.html>

<https://www.geeksforgeeks.org/python/python-pandas-map/>

Next Section:

 [4.1. Renaming Indexes and Columns](#)